

# 核心独占调度系统性能深度分析报告 (Science Style)

本报告针对**核心独占 (Core Pinning)** 模式下的高频任务调度器进行定量分析。我们通过追踪任务从“唤醒”到“休眠”的完整物理生命周期，评估系统在**确定性 (Determinism)**、**负载均衡**及**并行扩展性**方面的表现。

## 1. 全路径生命周期定义 (Full Lifecycle)

为了精确捕捉调度损耗，我们将每次任务执行拆解为以下五个阶段：

- Wakeup Latency ( $L_{wake}$ )**: 线程收到信号到开始从全局队列获取任务的物理延迟。
- Pack Overhead ( $L_{pack}$ )**: 任务分发、上下文切换及输入数据打包的开销。
- Effective Computation ( $C_i$ )**: 算法核心逻辑执行时间。**这是系统唯一的有效产出。**
- Post Overhead ( $L_{post}$ )**: 计算完成到结果路由、状态更新的开销。
- Sleep Latency ( $L_{sleep}$ )**: 任务结束后，线程返回调度循环并重新进入休眠的残余延迟。

## 2. 视觉规范说明

- 时间单位**: 宏观视图统一使用 **毫秒 (ms)**，微观分析使用 **微秒 (us)**。
- 配色方案**: 采用低饱和度学术色系。**深蓝灰 (#3B5998)** 代表有效计算，**暗红 (#A6192E)** 代表调度损耗。
- 数据过滤**: 针对抖动分析，提供 **P99 截断版**（展示稳定性）与 **全量版**（展示最坏情况执行时间 WCET）。

```
In [69]: import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
import re, os

from asttokens.util import is_slice

def _parse_hierarchy_data(csv):
    if not os.path.exists(csv):
        print(f"✗ File not found: {csv}")
        return pd.DataFrame()

    df = pd.read_csv(csv).sort_values(["tid", "seq"])
    df = df[df["t_us"] >= 0]

    excl = {"timer", "main::expand", "main::rollover"}
    rows = []
    warning_count = 0
    warning_details = []
```

```

# 确定全局起始时间, 用于计算毫秒偏移
t_min_global = df["t_us"].min()

for tid, g in df.groupby("tid"):
    events = g.reset_index(drop=True)
    for i in range(len(events)):
        curr = events.iloc[i]
        if curr["kind"] == "execute" and curr["tag"] not in excl:
            try:
                prev_s = events.iloc[:i]
                rel = prev_s[prev_s["kind"] == "release"].iloc[-1]
                wak = prev_s[prev_s["kind"] == "wake"].iloc[-1]
                next_s = events.iloc[i+1:]
                com = next_s[next_s["kind"] == "complete"].iloc[0]
                fin = next_s[next_s["kind"] == "finished"].iloc[0]
                slp = next_s[next_s["kind"] == "sleep"].iloc[0]
                # temp_1 = next_s[next_s["kind"] == "temp_1"].iloc[0]
                # temp_2 = next_s[next_s["kind"] == "temp_2"].iloc[0]

                # 按照用户要求分类计算:
                wakeup_lat = rel["t_us"] - wak["t_us"]
                pack = curr["t_us"] - rel["t_us"]
                algo_us = com["t_us"] - curr["t_us"]
                post = fin["t_us"] - com["t_us"]
                sleep_lat = slp["t_us"] - fin["t_us"]
                # sleep_lat_1 = temp_1["t_us"] - fin["t_us"]
                # sleep_lat_2 = temp_2["t_us"] - temp_1["t_us"]
                # sleep_lat_3 = slp["t_us"] - temp_2["t_us"]

                # === 警告逻辑: 检查五个值是否小于零 ===
                negative_values = {}
                if wakeup_lat < 0:
                    negative_values['wakeup_lat'] = wakeup_lat
                if pack < 0:
                    negative_values['pack'] = pack
                if algo_us < 0:
                    negative_values['algo_us'] = algo_us
                if post < 0:
                    negative_values['post'] = post
                if sleep_lat < 0:
                    negative_values['sleep_lat'] = sleep_lat

                if negative_values:
                    warning_count += 1
                    warning_details.append({
                        'tid': tid,
                        'algo': curr["tag"],
                        'seq': curr["seq"],
                        'negative_values': negative_values,
                        'timestamps': {
                            'wak': wak["t_us"],
                            'rel': rel["t_us"],
                            'curr': curr["t_us"],
                            'com': com["t_us"],

```

```

        'fin': fin["t_us"],
        'slp': slp["t_us"]
    }
})
# 打印警告信息
print(f"\n⚠ WARNING #{warning_count}: Negative val")
print(f"    Thread: {tid}, Algorithm: {curr['tag']},")
for key, val in negative_values.items():
    print(f"        {key}: {val:.2f} us (negative)")
print(f"    Timeline: wake={wak['t_us']:.2f} → rel={r
    f"curr={curr['t_us']:.2f} → com={com['t_us']:.
    f"fin={fin['t_us']:.2f} → slp={slp['t_us']:.2f
# === 警告逻辑结束 ===

rows.append({
    "tid": str(tid),
    "algo": curr["tag"],
    "algo_us": algo_us,
    # 甘特图核心字段: 起始点和持续时间 (ms)
    "start_ms": (wak["t_us"] - t_min_global) / 1000.0,
    "algo_ms": algo_us / 1000.0,
    # 统计字段
    "algo_us": algo_us,
    "wakeup_lat": wakeup_lat,
    "sleep_lat": sleep_lat,
    # "sleep_lat_1": sleep_lat_1,
    # "sleep_lat_2": sleep_lat_2,
    # "sleep_lat_3": sleep_lat_3,
    "pack": pack,
    "post": post,
    "pack_post_us": pack + post, # 算法级开销
    "total_extra_us": (rel["t_us"] - wak["t_us"]) + pack
    "total_cycle_us": slp["t_us"] - wak["t_us"]
})
except Exception as e:
    # 打印异常信息以便调试
    print(f"⚠ Exception at tid={tid}, i={i}: {e}")
    continue

# === 在函数结束前打印汇总信息 ===
print(f"\n{'='*60}")
print(f"📊 PARSING COMPLETE")
print(f"\n{'='*60}")
print(f"Total events parsed: {len(rows)}")
print(f"Total threads: {df['tid'].nunique()}")

if warning_count > 0:
    print(f"\n⚠ WARNINGS: {warning_count} negative value(s) detected!")
    print(f"    First 3 warnings:")
    for i, detail in enumerate(warning_details[:3]):
        print(f"        {i+1}. tid={detail['tid']}, algo={detail['algo']}, "
            f"negatives={list(detail['negative_values'].keys())}")
    if warning_count > 3:
        print(f"        ... and {warning_count - 3} more warnings")
else:
    print(f"\n✅ All values are non-negative. No warnings detected.")

```

```

print(f"{' '*60}\n")

return pd.DataFrame(rows)

# 执行解析
ROS2_TRACE = ("ros2_feedback_u10_m1_ms00_s20907657_145904_timeline.csv")
FINS_TRACE = ("fins_trace_feedback_u10_m1_ms00_s20907657.csv")

ros2_raw_df = pd.read_csv(ROS2_TRACE)
fins_raw_df = pd.read_csv(FINS_TRACE)

print(f" Raw data loaded: {len(ros2_raw_df)} rows")
print(f" Raw data loaded: {len(fins_raw_df)} rows")

ros2_analysis_df = _parse_hierarchy_data(ROS2_TRACE)
fins_analysis_df = _parse_hierarchy_data(FINS_TRACE)

```

Raw data loaded: 7696 rows  
Raw data loaded: 1776 rows

=====

 PARSING COMPLETE

=====

Total events parsed: 1480  
Total threads: 1

✓ All values are non-negative. No warnings detected.

=====

=====

 PARSING COMPLETE

=====

Total events parsed: 201  
Total threads: 3

✓ All values are non-negative. No warnings detected.

=====

## 一、宏观并行行为分析 (Timeline & Gantt)

### 1. 任务甘特图 (Hardware Core Gantt Chart)

展示有效计算块 ( $C_i$ ) 的持续时长。

- **分析重点:** 观察各核心条块之间的**横向间隙**。这些空隙（气泡）代表了调度器无法填满的资源空洞，是加速比无法达到线性的根本原因。

```

In [35]: def draw_px_gantt(df_jobs):
          if df_jobs.empty:
              print("Warning: Input DataFrame is empty.")

```

```

    return

# 1. 精选 Science 级低饱和度多色盘 (针对不同 Algo)
# 选色原则: 冷暖交替, 灰度梯度一致
DISCRETE_SCI_COLORS = [
    "#3B5998", # 灰蓝
    "#5E8B61", # 鼠尾草绿
    "#A6192E", # 砖红
    "#D6A531", # 暗金
    "#709BFF", # 浅钢蓝
    "#925E9F", # 灰紫
    "#0099B4", # 墨青
    "#FDAF91", # 柔粉
    "#4D4D4D", # 碳灰
    "#ADB6B6", # 铝灰
]

# 2. 线程 TID 自然排序 (确保 T0, T1, T2... 逻辑顺序)
def extract_num(s):
    n = re.findall(r'\d+', str(s))
    return int(n[0]) if n else 0

tids_sorted = sorted(df_jobs["tid"].unique(), key=extract_num)
# 算法也进行排序, 确保图例整齐
algos_sorted = sorted(df_jobs["algo"].unique(), key=extract_num)

# 3. 绘图
fig = px.bar(
    df_jobs,
    base="start_ms",      # 任务起点 (ms)
    x="algo_ms",          # 任务宽度 (ms)
    y="tid",
    color="algo",         # 区分不同算法
    orientation='h',
    category_orders={"tid": tids_sorted, "algo": algos_sorted},
    color_discrete_sequence=DISCRETE_SCI_COLORS,
    title="Hardware Core Execution Continuity (Science Style)",
    labels={"start_ms": "Timeline", "algo_ms": "Exec", "tid": "Thread ID"}
)

# 4. 论文级布局优化
fig.update_layout(
    plot_bgcolor='rgba(248, 248, 248, 1)', # 极浅灰背景, 使白色气泡更明显
    xaxis=dict(
        title="Time Offset (ms)",
        showgrid=True,
        gridcolor='white', # 白色网格在灰底上非常高级
        linecolor='black',
        ticks='outside',
        zeroline=False
    ),
    yaxis=dict(
        title="Exclusive Thread / Core",
        showgrid=False, # Y 轴不需网格, 保持横向视觉流
        autorange="reversed", # 线程 0 置顶
        linecolor='black',

```

```

        ticks='outside'
    ),
    font=dict(family="Arial", size=12, color="black"),
    height=350 + (len(tids_sorted) * 30), # 根据线程数动态调整高度
    margin=dict(l=100, r=30, t=100, b=80), # 增加左边距给 TID 标签
)

# 5. 交互与渲染优化
fig.update_traces(
    marker_line_width=0.5, # 给极其微小的任务块增加细边框以便识别
    marker_line_color="rgba(255,255,255,0.2)",
    opacity=0.9,
    # 悬浮窗：微秒级精度标注
    hovertemplate="<b>Core %y}</b><br>Algo: %y}<br>Start: %y}<br>End: %y}"
    text=df_jobs["algo"]
)

fig.show()

```

```
In [36]: draw_px_gantt(ros2_analysis_df)
```

```
In [37]: draw_px_gantt(fins_analysis_df)
```

```
In [38]: # 1. Nature Classic - 包含所有类别
NATURE_CLASSIC = {
    # 四个阶段
    'wakeup_lat': '#3B7A9E', # 沉稳蓝色
    'pack': '#6BA3B8', # 柔和蓝绿
    'algo_us': '#E8E8E8', # 【淡灰】 有效功设为背景
    'post': '#E8A87C', # 暖橙
    'sleep_lat': '#C87D7D', # 柔和红褐

    # 新增三类
    'Active': '#2C5F7A', # 深海蓝 - 活跃状态
    'Overhead': '#D4834A', # 橙褐 - 开销
    'Idle': '#8CAA8C', # 柔和灰绿 - 空闲状态
}

# 2. Nature Earth - 大地色系
NATURE_EARTH = {
    'wakeup_lat': '#4A6B7F', # 深蓝灰
    'pack': '#7A9B8C', # 苔藓绿
    'algo_us': '#E8E8E8', # 【淡灰】 有效功设为背景
    'post': '#C49A6C', # 沙褐色
    'sleep_lat': '#B57D7D', # 玫瑰棕

    'Active': '#3D5A6B', # 深灰蓝
    'Overhead': '#B8896C', # 暖棕
    'Idle': '#9AAB9A', # 灰
}

# 3. Nature Ocean - 海洋色系
NATURE_OCEAN = {
    'wakeup_lat': '#2C5F7A', # 深海蓝
    'pack': '#4A8C8C', # 碧绿

```

```

    'algo_us': '#E8E8E8',      # 【淡灰】 有效功设为背景
    'post': '#D4A76A',        # 金色
    'sleep_lat': '#B06A6A',   # 珊瑚红

    'Active': '#1A4A6A',      # 深蓝
    'Overhead': '#C4956A',    # 暖金褐
    'Idle': '#6A9A9A',        # 海绿
}

```

#### # 4. Nature Journal - Nature期刊常见配色

```

NATURE_JOURNAL = {
    'wakeup_lat': '#1F567D',   # 深蓝
    'pack': '#5B8C5A',         # 森林绿
    'algo_us': '#E8E8E8',      # 【淡灰】 有效功设为背景
    'post': '#D4834A',         # 橙褐
    'sleep_lat': '#B55A5A',    # 砖红

    'Active': '#1A4A6E',      # 墨水蓝
    'Overhead': '#C47A4A',     # 陶土橙
    'Idle': '#7A9A7A',        # 灰绿
}

```

#### # 5. Nature Minimal - 极简灰度

```

NATURE_MINIMAL = {
    'wakeup_lat': '#4A4A4A',   # 深灰
    'pack': '#7A7A7A',         # 中灰
    'algo_us': '#E8E8E8',      # 【淡灰】 有效功设为背景
    'post': '#AAAAAA',         # 浅灰
    'sleep_lat': '#D4D4D4',    # 极浅灰

    'Active': '#333333',       # 最深灰
    'Overhead': '#666666',     # 中深灰
    'Idle': '#999999',        # 中灰
}

```

#### # 6. Nature Garden - 花园色系, 温暖自然

```

NATURE_GARDEN = {
    'wakeup_lat': '#4A7A9C',   # 天蓝
    'pack': '#6A9B7A',         # 草绿
    'algo_us': '#E8E8E8',      # 【淡灰】 有效功设为背景
    'post': '#D4A06A',         # 麦黄
    'sleep_lat': '#C07A7A',    # 粉红

    'Active': '#3A6A8A',       # 深天蓝
    'Overhead': '#C4906A',     # 卡其
    'Idle': '#8AAA8A',         # 嫩绿
}

```

#### # 7. Nature Evening - 晚间色系, 沉稳

```

NATURE_EVENING = {
    'wakeup_lat': '#3D5A7A',   # 暮蓝
    'pack': '#5A7A6A',         # 暮绿
    'algo_us': '#E8E8E8',      # 【淡灰】 有效功设为背景
    'post': '#B08A6A',         # 暮橙
    'sleep_lat': '#A06A6A',    # 暮红
}

```

```

    'Active': '#2A4A6A',      # 夜蓝
    'Overhead': '#A07A5A',    # 暮褐
    'Idle': '#6A8A7A',        # 暮灰绿
}

PLA = NATURE_CLASSIC

```

## 二、核心资源利用效率分析 (Efficiency Breakdown)

本图表将单核独占的物理时间归一化为 **100%**，并切分为三个互斥状态：

1. **Active (有效功)**: 真正的算法产出时间占比。
2. **Sched Overhead (调度开销)**: 全路径 (Wake+Pack+Post+Sleep) 的非生产性耗时总和。
3. **Wait/Idle (空闲气泡)**: 核心处于 `cv_wait` 状态，等待任务流或同步信号的时间。

```

In [63]: import re
import plotly.express as px
import pandas as pd

def draw_thread_analysis_mixed(df_raw, df_jobs):
    """
    修正版：
    1. 利用率堆叠柱状图：横向 (Horizontal)
    2. 额外开销小提琴图：纵向 (Vertical)
    """
    if df_jobs.empty:
        print("数据为空")
        return

    # --- 1. 数据准备：利用率百分比 ---
    wall_clock = df_raw['t_us'].max() - df_raw['t_us'].min()
    thread_res = []

    for tid in df_jobs['tid'].unique():
        sub = df_jobs[df_jobs['tid'] == tid]
        active_time = sub['algo_us'].sum()
        overhead_time = sub['total_extra_us'].sum()
        # Idle = 总时间 - (算法时间 + 调度总开销)
        idle_time = max(0, wall_clock - (active_time + overhead_time))

        thread_res.append({"tid": tid, "State": "1. Active", "Value": (active_time / wall_clock) * 100})
        thread_res.append({"tid": tid, "State": "2. Overhead", "Value": (overhead_time / wall_clock) * 100})
        thread_res.append({"tid": tid, "State": "3. Idle", "Value": (idle_time / wall_clock) * 100})

    df_util = pd.DataFrame(thread_res)
    # 线程号自然排序 (T0, T1, T2...)
    tids_sorted = sorted(df_jobs["tid"].unique(), key=lambda x: int(re.findall(r'T\d+', str(x))[0]))

    # --- 2. 绘制【横向】堆叠利用率图 ---
    fig_util = px.bar(
        df_util,
        x="Value",          # X轴是百分比数值
        y="tid",
        title="Thread Utilization Breakdown",
        labels={"Value": "Percentage (%)", "tid": "Thread ID"},
        color="State",
        barmode="stack",
        orientation="horizontal",
    )

```

```

        y="tid",          # Y轴是线程ID
        color="State",
        orientation='h', # 横向
        category_orders={"tid": tids_sorted},
        color_discrete_map={
            "1. Active": PLA["Active"],
            "2. Overhead": PLA["Overhead"],
            "3. Idle": PLA["Idle"]
        },
        title="Thread-level Core Utilization Breakdown",
        labels={"Value": "Percentage of Time (%)", "tid": "Thread ID"}
    )

    fig_util.update_layout(
        plot_bgcolor='white',
        barmode='stack',
        xaxis=dict(range=[0, 105], gridcolor='#F0F0F0', ticksuffix="%", title="Percentage of Time (%)"),
        yaxis=dict(autorange="reversed", linecolor='black', title="Thread ID", ticksuffix=""),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right",
                    height=300 + (len(tids_sorted) * 30))
    )
    fig_util.show()

```

```
In [40]: draw_thread_analysis_mixed(ros2_raw_df, ros2_analysis_df)
```

```
In [41]: draw_thread_analysis_mixed(fins_raw_df, fins_analysis_df)
```

### 三、各算法的唤醒、包装、分发效率分析

1. 手风琴式堆叠柱状图 (Accordion Stacked Bar Chart): 展示每个算法的唤醒、包装、分发、计算、后处理、休眠的时间占比。

```

In [48]: import re
import plotly.express as px
import pandas as pd
import numpy as np

def draw_px_accordion_enhanced_us(df_jobs):
    """
    强化开销显示的风琴图（单位回归为 us）
    """
    if df_jobs.empty:
        print("数据为空")
        return

    # 1. 均值计算（直接使用原始 us 单位）
    cols = ["wakeup_lat", "pack", "algo_us", "post", "sleep_lat"]
    g = df_jobs.groupby("algo")[cols].mean().reset_index()

    # 2. 自然排序 (n0, n1, n2...)
    def extract_num(s):
        n = re.findall(r'\d+', str(s))

```

```

        return int(n[0]) if n else 0
g['sort_idx'] = g['algo'].apply(extract_num)
g = g.sort_values("sort_idx", ascending=False)

# 3. 准备百分比数据 (归一化)
g_pct = g.copy()
g_pct['total'] = g_pct[cols].sum(axis=1)
for col in cols:
    g_pct[col] = (g_pct[col] / g_pct['total']) * 100

# --- 绘图 A: 绝对耗时构成 (us) ---
long_abs = g.melt(id_vars="algo", value_vars=cols, var_name="Stage", val
fig_abs = px.bar(
    long_abs, y="algo", x="us", color="Stage",
    orientation='h',
    color_discrete_map=PLA,
    title="Mean Task Lifecycle Breakdown (Absolute us)",
    labels={"us": "Time Duration (us)", "algo": "Algorithm Node"}
)

# 移除右侧的 Sched Overhead 标注
# for i in range(len(g)):
#     row = g.iloc[i]
#     overhead = row["wakeup_lat"] + row["pack"] + row["post"] + row["sl
#     fig_abs.add_annotation(
#         x=row[cols].sum(),
#         y=row["algo"],
#         text=f" Sched:{overhead:.1f}us",
#         showarrow=False,
#         xanchor="left",
#         font=dict(color="#A6192E", size=10, family="Arial Black")
#     )

fig_abs.update_layout(
    plot_bgcolor='white',
    barmode='stack',
    height=len(g)*35+150,
    xaxis=dict(gridcolor='#F0F0F0', title="Average Time (us)")
)
fig_abs.show()

# --- 绘图 B: 结构占比构成 (100% 归一化) ---
long_pct = g_pct.melt(id_vars="algo", value_vars=cols, var_name="Stage",
fig_pct = px.bar(
    long_pct, y="algo", x="pct", color="Stage",
    orientation='h',
    color_discrete_map=PLA,
    title="Task Lifecycle Structural Ratio (Normalized %)",
    labels={"pct": "Composition Ratio (%)", "algo": "Algorithm Node"}
    # 删除 text_auto='.1f' 这行, 不显示数据标签
)

fig_pct.update_layout(
    plot_bgcolor='white',
    barmode='stack',
    xaxis=dict(title="Percentage (%)", range=[0, 100], ticksuffix="%", g

```

```

        height=len(g)*35+150,
        legend_title="Lifecycle Stages"
    )
    fig_pct.show()

```

```

In [43]: # 调用
draw_px_accordion_enhanced_us(ros2_analysis_df)

```

```

In [44]: # 调用
draw_px_accordion_enhanced_us(fins_analysis_df)

```

## 四、额外开销的真实分布

1. **小提琴图 (Violin Plot)**: 展示每个线程的调度开销分布，重点关注均值。
2. **小提琴图 (Violin Plot)**: 展示每个算法的调度开销分布，重点关注均值。分别提供全量与 p99 截断两种视图，便于分析调度器的稳定性与最坏情况执行时间 (WCET)。

```

In [47]: def draw_system_summary_violin(df_jobs):
# 创建统一标签
df_sum = df_jobs.copy()
df_sum["System"] = "All Tasks"

fig = px.violin(df_sum, x="System", y="total_extra_us",
                box=True, points="outliers",
                title="System-wide Total Extra Overhead Distribution",
                color_discrete_sequence=[PLA["Overhead"]])

# 增加均值和 P99 标注
mean_val = df_sum["total_extra_us"].mean()
p99_val = df_sum["total_extra_us"].quantile(0.99)

fig.add_hline(y=mean_val, line_dash="dot", annotation_text=f"Mean: {mean_val:.1f}us", annotation_position="top")
fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
              annotation_text=f"P99: {p99_val:.1f}us", annotation_position="top")
fig.update_yaxes(range=[0, None]) # 从0开始
fig.update_layout(plot_bgcolor='white', yaxis_title="Total Extra Overhead")
fig.show()

```

```

In [46]: draw_system_summary_violin(ros2_analysis_df)

```

```

In [45]: draw_system_summary_violin(fins_analysis_df)

```

```

In [58]: # Cell 1: Wakeup Latency Distribution by Algorithm
def draw_algo_wakeup_violin(df, p99_cut=False):
    """
    Draw algorithm-level Wakeup Latency distribution
    Wakeup Latency: wake → release
    """
    metric = 'wakeup_lat'
    metric_name = 'Wakeup Latency (wake→release)'

```

```

df_plot = df.copy()

negative_mask = df_plot[metric] < 0
if negative_mask.any():
    num_negative = negative_mask.sum()
    negative_samples = df_plot[negative_mask][metric].tolist()
    print(f"ERROR: Found {num_negative} negative value(s) in {metric}:")
    print(f"Negative values: {negative_samples[:10]}{'...' if num_negati")
    raise ValueError(f"Negative values detected in {metric}. Please chec

if p99_cut:
    p99_threshold = df_plot[metric].quantile(0.99)
    df_plot = df_plot[df_plot[metric] <= p99_threshold]
    suffix = " (P99 Truncated)"
else:
    suffix = ""

fig = px.violin(
    df_plot,
    x='algo',
    y=metric,
    box=True,
    points="outliers",
    title=f"Algorithm {metric_name} Distribution{suffix}",
    color_discrete_sequence=[PLA['wakeup_lat']], # 直接取单个颜色, 注意是列
    labels={'algo': 'Algorithm', metric: f'{metric_name} (μs)'}
)

mean_val = df_plot[metric].mean()
median_val = df_plot[metric].median()
p99_val = df_plot[metric].quantile(0.99)

fig.add_hline(y=mean_val, line_dash="dot", line_color="black",
              annotation_text=f"Mean: {mean_val:.1f}μs", annotation_posi
fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
              annotation_text=f"P99: {p99_val:.1f}μs", annotation_positi
fig.add_hline(y=median_val, line_dash="dashdot", line_color="blue",
              annotation_text=f"Median: {median_val:.1f}μs", annotation_

fig.update_layout(plot_bgcolor='white', height=500, width=900)
fig.update_yaxes(gridcolor='lightgray', gridwidth=0.5, zeroline=True, ze
fig.show()

print(f"\nWakeup Latency - Mean: {mean_val:.1f}μs, P99: {p99_val:.1f}μs,

```

```
In [50]: draw_algo_wakeup_violin(ros2_analysis_df, p99_cut=False)
```

Wakeup Latency - Mean: 783.1μs, P99: 3120.4μs, Samples: 1480

```
In [51]: draw_algo_wakeup_violin(fins_analysis_df, p99_cut=False)
```

Wakeup Latency - Mean: 1.2μs, P99: 8.0μs, Samples: 201

```
In [53]: # Cell 2: Pack Overhead Distribution by Algorithm
```

```

def draw_algo_pack_violin(df, p99_cut=False):
    """
    Draw algorithm-level Pack Overhead distribution

```

```

Pack: release → execute
"""

metric = 'pack'
metric_name = 'Pack Overhead (release→execute)'

df_plot = df.copy()

negative_mask = df_plot[metric] < 0
if negative_mask.any():
    num_negative = negative_mask.sum()
    negative_samples = df_plot[negative_mask][metric].tolist()
    print(f"ERROR: Found {num_negative} negative value(s) in {metric}:")
    print(f"Negative values: {negative_samples[:10]}{'...' if num_negative > 10 else ''}")
    raise ValueError(f"Negative values detected in {metric}. Please check the data.")

if p99_cut:
    p99_threshold = df_plot[metric].quantile(0.99)
    df_plot = df_plot[df_plot[metric] <= p99_threshold]
    suffix = " (P99 Truncated)"
else:
    suffix = ""

fig = px.violin(
    df_plot,
    x='algo',
    y=metric,
    box=True,
    points="outliers",
    title=f"Algorithm {metric_name} Distribution{suffix}",
    color_discrete_sequence=[PLA['pack']], # 直接取单个颜色, 注意是列表
    labels={'algo': 'Algorithm', metric: f'{metric_name} (μs)'}
)

mean_val = df_plot[metric].mean()
median_val = df_plot[metric].median()
p99_val = df_plot[metric].quantile(0.99)

fig.add_hline(y=mean_val, line_dash="dot", line_color="black",
              annotation_text=f"Mean: {mean_val:.1f}μs", annotation_position="top")
fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
              annotation_text=f"P99: {p99_val:.1f}μs", annotation_position="top")
fig.add_hline(y=median_val, line_dash="dashdot", line_color="blue",
              annotation_text=f"Median: {median_val:.1f}μs", annotation_position="top")

fig.update_layout(plot_bgcolor='white', height=500, width=900)
fig.update_yaxes(gridcolor='lightgray', gridwidth=0.5, zeroline=True, zero=0)
fig.show()

print(f"\nPack Overhead - Mean: {mean_val:.1f}μs, P99: {p99_val:.1f}μs, Samples: {len(df_plot)}")

```

```
In [54]: draw_algo_pack_violin(ros2_analysis_df, p99_cut=False)
```

Pack Overhead - Mean: 788.6μs, P99: 4183.6μs, Samples: 1480

```
In [55]: draw_algo_pack_violin(fins_analysis_df, p99_cut=False)
```

Pack Overhead - Mean: 2.8 $\mu$ s, P99: 12.0 $\mu$ s, Samples: 201

```
In [59]: # Cell 3: Post Overhead Distribution by Algorithm
def draw_algo_post_violin(df, p99_cut=False):
    """
    Draw algorithm-level Post Overhead distribution
    Post: complete → finished
    """
    metric = 'post'
    metric_name = 'Post Overhead (complete→finished)'

    df_plot = df.copy()

    negative_mask = df_plot[metric] < 0
    if negative_mask.any():
        num_negative = negative_mask.sum()
        negative_samples = df_plot[negative_mask][metric].tolist()
        print(f"ERROR: Found {num_negative} negative value(s) in {metric}:")
        print(f"Negative values: {negative_samples[:10]}{'...' if num_negative > 10 else ''}")
        raise ValueError(f"Negative values detected in {metric}. Please check the data.")

    if p99_cut:
        p99_threshold = df_plot[metric].quantile(0.99)
        df_plot = df_plot[df_plot[metric] <= p99_threshold]
        suffix = " (P99 Truncated)"
    else:
        suffix = ""

    fig = px.violin(
        df_plot,
        x='algo',
        y=metric,
        box=True,
        points="outliers",
        title=f"Algorithm {metric_name} Distribution{suffix}",
        color_discrete_sequence=[PLA['post']], # 直接取单个颜色, 注意是列表
        labels={'algo': 'Algorithm', metric: f'{metric_name} ( $\mu$ s)'}
    )

    mean_val = df_plot[metric].mean()
    median_val = df_plot[metric].median()
    p99_val = df_plot[metric].quantile(0.99)

    fig.add_hline(y=mean_val, line_dash="dot", line_color="black",
                  annotation_text=f"Mean: {mean_val:.1f} $\mu$ s", annotation_position="top")
    fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
                  annotation_text=f"P99: {p99_val:.1f} $\mu$ s", annotation_position="top")
    fig.add_hline(y=median_val, line_dash="dashdot", line_color="blue",
                  annotation_text=f"Median: {median_val:.1f} $\mu$ s", annotation_position="top")

    fig.update_layout(plot_bgcolor='white', height=500, width=900)
    fig.update_yaxes(gridcolor='lightgray', gridwidth=0.5, zeroline=True, zero=0)
    fig.show()

    print(f"\nPost Overhead - Mean: {mean_val:.1f} $\mu$ s, P99: {p99_val:.1f} $\mu$ s,
```

```
In [60]: draw_algo_post_violin(ros2_analysis_df, p99_cut=False)
```

Post Overhead - Mean: 5.1 $\mu$ s, P99: 27.9 $\mu$ s, Samples: 1480

```
In [61]: draw_algo_post_violin(fins_analysis_df, p99_cut=False)
```

Post Overhead - Mean: 3.3 $\mu$ s, P99: 13.0 $\mu$ s, Samples: 201

```
In [ ]: # Cell 4: Sleep Latency Distribution by Algorithm
```

```
def draw_algo_sleep_violin(df, p99_cut=False):
    """
    Draw algorithm-level Sleep Latency distribution
    Sleep Latency: finished  $\rightarrow$  sleep
    """
    metric = 'sleep_lat'
    metric_name = 'Sleep Latency (finished $\rightarrow$ sleep)'

    df_plot = df.copy()

    negative_mask = df_plot[metric] < 0
    if negative_mask.any():
        num_negative = negative_mask.sum()
        negative_samples = df_plot[negative_mask][metric].tolist()
        print(f"ERROR: Found {num_negative} negative value(s) in {metric}:")
        print(f"Negative values: {negative_samples[:10]}{'...' if num_negative > 10 else ''}")
        raise ValueError(f"Negative values detected in {metric}. Please check the data.")

    if p99_cut:
        p99_threshold = df_plot[metric].quantile(0.99)
        df_plot = df_plot[df_plot[metric] <= p99_threshold]
        suffix = " (P99 Truncated)"
    else:
        suffix = ""

    fig = px.violin(
        df_plot,
        x='algo',
        y=metric,
        box=True,
        points="outliers",
        title=f"Algorithm {metric_name} Distribution{suffix}",
        color_discrete_sequence=[PLA['sleep_lat']], # 直接取单个颜色, 注意是列表
        labels={'algo': 'Algorithm', 'metric': f'{metric_name} ( $\mu$ s)'}
    )

    mean_val = df_plot[metric].mean()
    median_val = df_plot[metric].median()
    p99_val = df_plot[metric].quantile(0.99)

    fig.add_hline(y=mean_val, line_dash="dot", line_color="black",
                  annotation_text=f"Mean: {mean_val:.1f} $\mu$ s", annotation_posi
    fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
                  annotation_text=f"P99: {p99_val:.1f} $\mu$ s", annotation_positi
    fig.add_hline(y=median_val, line_dash="dashdot", line_color="blue",
                  annotation_text=f"Median: {median_val:.1f} $\mu$ s", annotation_
```

```
fig.update_layout(plot_bgcolor='white', height=500, width=900)
fig.update_yaxes(gridcolor='lightgray', gridwidth=0.5, zeroline=True, ze
fig.show()

print(f"\nSleep Latency - Mean: {mean_val:.1f}μs, P99: {p99_val:.1f}μs,
```

```
In [ ]: draw_algo_sleep_violin(ros2_analysis_df, p99_cut=False)
```

```
In [ ]: draw_algo_sleep_violin(fins_analysis_df, p99_cut=False)
```

```
In [68]: # Cell 4: Sleep Latency 1 Distribution by Algorithm
def draw_algo_sleep_part1_violin(df, p99_cut=False):
    """
    Draw algorithm-level Sleep Latency distribution
    Sleep Latency: finished → sleep
    """
    metric = 'sleep_lat_1'
    metric_name = 'Sleep Latency (finished→sleep)'

    df_plot = df.copy()

    negative_mask = df_plot[metric] < 0
    if negative_mask.any():
        num_negative = negative_mask.sum()
        negative_samples = df_plot[negative_mask][metric].tolist()
        print(f"ERROR: Found {num_negative} negative value(s) in {metric}:")
        print(f"Negative values: {negative_samples[:10]}{'...' if num_negative > 10 else ''}")
        raise ValueError(f"Negative values detected in {metric}. Please check")

    if p99_cut:
        p99_threshold = df_plot[metric].quantile(0.99)
        df_plot = df_plot[df_plot[metric] <= p99_threshold]
        suffix = " (P99 Truncated)"
    else:
        suffix = ""

    fig = px.violin(
        df_plot,
        x='algo',
        y=metric,
        box=True,
        points="outliers",
        title=f"Algorithm {metric_name} Distribution{suffix}",
        color_discrete_sequence=[PLA['sleep_lat']], # 直接取单个颜色, 注意是列表
        labels={'algo': 'Algorithm', metric: f'{metric_name} (μs)'}
    )

    mean_val = df_plot[metric].mean()
    median_val = df_plot[metric].median()
    p99_val = df_plot[metric].quantile(0.99)

    fig.add_hline(y=mean_val, line_dash="dot", line_color="black",
                  annotation_text=f"Mean: {mean_val:.1f}μs", annotation_posi
    fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
                  annotation_text=f"P99: {p99_val:.1f}μs", annotation_positi
```

```

fig.add_hline(y=median_val, line_dash="dashdot", line_color="blue",
              annotation_text=f"Median: {median_val:.1f}μs", annotation_posi

fig.update_layout(plot_bgcolor='white', height=500, width=900)
fig.update_yaxes(gridcolor='lightgray', gridwidth=0.5, zeroline=True, ze
fig.show()

print(f"\nSleep Latency 1 - Mean: {mean_val:.1f}μs, P99: {p99_val:.1f}μs")

def draw_algo_sleep_part2_violin(df, p99_cut=False):
    """
    Draw algorithm-level Sleep Latency distribution
    Sleep Latency: finished → sleep
    """
    metric = 'sleep_lat_2'
    metric_name = 'Sleep Latency (finished→sleep)'

    df_plot = df.copy()
    if p99_cut:
        p99_threshold = df_plot[metric].quantile(0.99)
        df_plot = df_plot[df_plot[metric] <= p99_threshold]
        suffix = " (P99 Truncated)"
    else:
        suffix = ""

    fig = px.violin(
        df_plot,
        x='algo',
        y=metric,
        box=True,
        points="outliers",
        title=f"Algorithm {metric_name} Distribution{suffix}",
        color_discrete_sequence=[PLA['sleep_lat']], # 直接取单个颜色, 注意是列表
        labels={'algo': 'Algorithm', metric: f'{metric_name} (μs)'}
    )

    mean_val = df_plot[metric].mean()
    median_val = df_plot[metric].median()
    p99_val = df_plot[metric].quantile(0.99)

    fig.add_hline(y=mean_val, line_dash="dot", line_color="black",
                  annotation_text=f"Mean: {mean_val:.1f}μs", annotation_posi
    fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
                  annotation_text=f"P99: {p99_val:.1f}μs", annotation_positi
    fig.add_hline(y=median_val, line_dash="dashdot", line_color="blue",
                  annotation_text=f"Median: {median_val:.1f}μs", annotation_

    fig.update_layout(plot_bgcolor='white', height=500, width=900)

    fig.update_yaxes(gridcolor='lightgray', gridwidth=0.5, zeroline=True, ze
    fig.show()

    print(f"\nSleep Latency 2 - Mean: {mean_val:.1f}μs, P99: {p99_val:.1f}μs")

```

```

def draw_algo_sleep_part3_violin(df, p99_cut=False):
    """
    Draw algorithm-level Sleep Latency distribution
    Sleep Latency: finished → sleep
    """
    metric = 'sleep_lat_3'
    metric_name = 'Sleep Latency (finished→sleep)'

    df_plot = df.copy()
    if p99_cut:
        p99_threshold = df_plot[metric].quantile(0.99)
        df_plot = df_plot[df_plot[metric] <= p99_threshold]
        suffix = " (P99 Truncated)"
    else:
        suffix = ""

    fig = px.violin(
        df_plot,
        x='algo',
        y=metric,
        box=True,
        points="outliers",
        title=f"Algorithm {metric_name} Distribution{suffix}",
        color_discrete_sequence=[PLA['sleep_lat']], # 直接取单个颜色, 注意是列表
        labels={'algo': 'Algorithm', metric: f'{metric_name} (μs)'}
    )

    mean_val = df_plot[metric].mean()
    median_val = df_plot[metric].median()
    p99_val = df_plot[metric].quantile(0.99)

    fig.add_hline(y=mean_val, line_dash="dot", line_color="black",
                  annotation_text=f"Mean: {mean_val:.1f}μs", annotation_posi
    fig.add_hline(y=p99_val, line_dash="dash", line_color="red",
                  annotation_text=f"P99: {p99_val:.1f}μs", annotation_positi
    fig.add_hline(y=median_val, line_dash="dashdot", line_color="blue",
                  annotation_text=f"Median: {median_val:.1f}μs", annotation_

    fig.update_layout(plot_bgcolor='white', height=500, width=900)
    fig.update_yaxes(gridcolor='lightgray', gridwidth=0.5, zeroline=True, ze
    fig.show()

    print(f"\nSleep Latency 3 - Mean: {mean_val:.1f}μs, P99: {p99_val:.1f}μs

```

```

In [ ]: # draw_algo_sleep_part1_violin(fins_analysis_df)
        # draw_algo_sleep_part2_violin(fins_analysis_df)
        # draw_algo_sleep_part3_violin(fins_analysis_df)

```

## 五、任务负载结构分析 (Structural Composition)

本部分通过双维度指标评估任务集的特征：

- **Task Frequency Ratio ( $P_i$ ):** 任务发生的频次分布。高频小任务往往是调度开销 (Overhead) 累积的主要来源。

- **Effective Workload Ratio ( $P_i \times C_i$ ):** 任务对 CPU 总时间的实际占用贡献。用于识别系统中的“重型节点”，即 Amahdl 定律中的串行瓶颈或关键路径。

```
In [64]: import plotly.graph_objects as go
from plotly.subplots import make_subplots
import pandas as pd
import re

# 使用你定义的调色板
PLA = NATURE_CLASSIC

def draw_task_structural_composition_donut(df_jobs):
    """
    绘制任务负载结构分析的双环图
    - 左环: Task Frequency Ratio ( $P_i$ ) - 任务发生频次分布
    - 右环: Effective Workload Ratio ( $P_i \times C_i$ ) - 任务对CPU总时间的实际占用贡献
    """
    if df_jobs.empty:
        print("数据为空")
        return

    # 1. 计算每个算法的统计指标
    freq = df_jobs.groupby('algo').size().reset_index(name='count')
    total_jobs = len(df_jobs)
    freq['P_i'] = freq['count'] / total_jobs * 100

    total_time = df_jobs.groupby('algo')['algo_us'].sum().reset_index(name='total_time')
    total_cpu_time = df_jobs['algo_us'].sum()

    df_stats = freq.merge(total_time, on='algo')
    df_stats['C_i_ratio'] = df_stats['total_time'] / total_cpu_time * 100
    df_stats['P_i_times_C_i'] = df_stats['P_i'] * df_stats['C_i_ratio'] / 100

    # 提取数字用于自然排序
    def extract_num(s):
        n = re.findall(r'\d+', str(s))
        return int(n[0]) if n else 0

    df_stats['sort_idx'] = df_stats['algo'].apply(extract_num)
    df_stats = df_stats.sort_values('sort_idx', ascending=True)

    # 2. 颜色分类函数
    def get_category(row):
        p_i = row['P_i']
        c_i = row['C_i_ratio']
        if p_i > 5 and c_i > 5:
            return 'critical'
        elif p_i > 5 and c_i <= 5:
            return 'overhead'
        elif p_i <= 5 and c_i > 5:
            return 'compute'
        else:
            return 'minor'

    # 颜色映射
```

```

category_colors = {
    'critical': PLA['Active'],      # '#2C5F7A' 深海蓝
    'overhead': PLA['Overhead'],   # '#D4834A' 橙褐
    'compute': PLA['sleep_lat'],    # '#C87D7D' 柔和红褐
    'minor': PLA['Idle']            # '#8CAA8C' 柔和灰绿
}

category_names = {
    'critical': 'Critical Path (High Freq + High Load)',
    'overhead': 'Overhead Source (High Freq + Low Load)',
    'compute': 'Compute Intensive (Low Freq + High Load)',
    'minor': 'Minor Tasks (Low Freq + Low Load)'
}

df_stats['category'] = df_stats.apply(get_category, axis=1)
df_stats['color'] = df_stats['category'].map(category_colors)

# 按类别分组排序
category_order = {'critical': 0, 'overhead': 1, 'compute': 2, 'minor': 3}
df_stats['cat_order'] = df_stats['category'].map(category_order)
df_stats = df_stats.sort_values(['cat_order', 'sort_idx'])

# 创建 hover 文本
hover_text_freq = []
hover_text_workload = []
for _, row in df_stats.iterrows():
    freq_text = (
        f"<b>{row['algo']}

```

```

        go.Pie(
            labels=df_stats['algo'],
            values=df_stats['P_i'],
            hole=0.5,
            marker=dict(
                colors=df_stats['color'],
                line=dict(color='white', width=2)
            ),
            textinfo='label+percent',
            textposition='outside',
            textfont=dict(size=11, color='#2C3E50'),
            hoverinfo='text',
            hovertext=hover_text_freq,
            showlegend=False,
            pull=[0.03 if row['category'] in ['critical', 'overhead'] else 0],
            sort=False
        ),
        row=1, col=1
    )

# ---- 右环图: 有效负载贡献 ( $P_i \times C_i$ ) ----
fig.add_trace(
    go.Pie(
        labels=df_stats['algo'],
        values=df_stats['P_i_times_C_i'],
        hole=0.5,
        marker=dict(
            colors=df_stats['color'],
            line=dict(color='white', width=2)
        ),
        textinfo='label+percent',
        textposition='outside',
        textfont=dict(size=11, color='#2C3E50'),
        hoverinfo='text',
        hovertext=hover_text_workload,
        showlegend=False,
        pull=[0.03 if row['category'] in ['critical', 'compute'] else 0],
        sort=False
    ),
    row=1, col=2
)

# ---- 图例 (使用 Scatter 模拟) ----
for cat, color in category_colors.items():
    fig.add_trace(
        go.Scatter(
            x=[None],
            y=[None],
            mode='markers',
            marker=dict(size=14, color=color, line=dict(color='white', w
            legendgroup=cat,
            showlegend=True,
            name=category_names[cat]
        )
    )

```

```

# ---- 更新布局 ----
fig.update_layout(
    plot_bgcolor='white',
    height=650,
    width=1100,
    title=dict(
        text='Task Load Structural Analysis',
        font=dict(size=20, family='Arial Black', color='#2C3E50'),
        x=0.5
    ),
    legend=dict(
        title=dict(
            text='<b>Task Category</b>',
            font=dict(size=13, color='#2C3E50')
        ),
        orientation='h',
        yanchor='bottom',
        y=-0.12,
        xanchor='center',
        x=0.5,
        bgcolor='rgba(255,255,255,0.95)',
        bordercolor='rgba(0,0,0,0.15)',
        borderwidth=1,
        font=dict(size=11, color='#2C3E50'),
        itemwidth=30,
        tracegroupgap=5
    ),
    font=dict(family='Arial, sans-serif', size=12),
    margin=dict(l=60, r=60, t=80, b=120),
    # 隐藏所有 x/y 轴
    xaxis=dict(showticklabels=False, showgrid=False, zeroline=False),
    yaxis=dict(showticklabels=False, showgrid=False, zeroline=False),
    xaxis2=dict(showticklabels=False, showgrid=False, zeroline=False),
    yaxis2=dict(showticklabels=False, showgrid=False, zeroline=False),
    annotations=[
        dict(
            text="P<sub>i</sub>",
            x=0.25,
            y=0.5,
            font=dict(size=14, color='#34495E', family='Arial Black'),
            showarrow=False
        ),
        dict(
            text="P<sub>i</sub> × C<sub>i</sub>",
            x=0.75,
            y=0.5,
            font=dict(size=14, color='#34495E', family='Arial Black'),
            showarrow=False
        )
    ]
)

fig.show()

# ---- 打印统计摘要 ----
print("\n" + "="*70)

```

```

print("📊 Task Load Structural Analysis Summary")
print("="*70)
print(f"Total Jobs: {total_jobs:,}")
print(f"Total CPU Time: {total_cpu_time:.2f} μs")
print(f"Number of Algorithm Types: {len(df_stats)}")

print("\n" + "-"*70)
print("🔥 Task Category Breakdown:")
print("-"*70)

categories_display = {
    'critical': '🔴 Critical Path (High Freq + High Load)',
    'overhead': '🟡 Overhead Source (High Freq + Low Load)',
    'compute': '🟠 Compute Intensive (Low Freq + High Load)',
    'minor': '🟢 Minor Tasks (Low Freq + Low Load)'
}

for cat, name in categories_display.items():
    df_cat = df_stats[df_stats['category'] == cat]
    if not df_cat.empty:
        total_p = df_cat['P_i'].sum()
        total_c = df_cat['C_i_ratio'].sum()
        print(f"\n{name}:")
        print(f"  Tasks: {'', '.join(df_cat['algo'].tolist())}")
        print(f"  Count: {len(df_cat)}")
        print(f"  Total Freq: {total_p:.1f}%")
        print(f"  Total CPU: {total_c:.1f}%")

print("\n" + "="*70)
print("💡 Interpretation:")
print("  • Critical Path: Serial bottleneck per Amdahl's Law")
print("  • Overhead Source: High-frequency small tasks")
print("  • Compute Intensive: Good candidates for parallelization")
print("  • Minor Tasks: Negligible impact on performance")
print("="*70 + "\n")

return df_stats

```

In [66]: `draw_task_structural_composition_donut(ros2_analysis_df)`

## Task Load Structural Analysis Summary

Total Jobs: 1,480  
Total CPU Time: 1485486.47  $\mu$ s  
Number of Algorithm Types: 10

### Task Category Breakdown:

#### Critical Path (High Freq + High Load):

Tasks: n0, n4, n5, n6, n7, n8, n9  
Count: 7  
Total Freq: 70.0%  
Total CPU: 89.9%

#### Overhead Source (High Freq + Low Load):

Tasks: n1, n2, n3  
Count: 3  
Total Freq: 30.0%  
Total CPU: 10.1%

#### Interpretation:

- Critical Path: Serial bottleneck per Amdahl's Law
- Overhead Source: High-frequency small tasks
- Compute Intensive: Good candidates for parallelization
- Minor Tasks: Negligible impact on performance

| Out[66]: | algo | count | P_i  | total_time | C_i_ratio | P_i_times_C_i | sort_idx | category | colo    |
|----------|------|-------|------|------------|-----------|---------------|----------|----------|---------|
| 0        | n0   | 148   | 10.0 | 612164.045 | 41.209668 | 4.120967      | 0        | critical | #2C5F7, |
| 4        | n4   | 148   | 10.0 | 104696.238 | 7.047943  | 0.704794      | 4        | critical | #2C5F7, |
| 5        | n5   | 148   | 10.0 | 74669.708  | 5.026617  | 0.502662      | 5        | critical | #2C5F7, |
| 6        | n6   | 148   | 10.0 | 169367.452 | 11.401481 | 1.140148      | 6        | critical | #2C5F7, |
| 7        | n7   | 148   | 10.0 | 89014.870  | 5.992304  | 0.599230      | 7        | critical | #2C5F7, |
| 8        | n8   | 148   | 10.0 | 93439.828  | 6.290184  | 0.629018      | 8        | critical | #2C5F7, |
| 9        | n9   | 148   | 10.0 | 192260.829 | 12.942617 | 1.294262      | 9        | critical | #2C5F7, |
| 1        | n1   | 148   | 10.0 | 43477.927  | 2.926848  | 0.292685      | 1        | overhead | #D4834, |
| 2        | n2   | 148   | 10.0 | 73592.228  | 4.954083  | 0.495408      | 2        | overhead | #D4834, |
| 3        | n3   | 148   | 10.0 | 32803.342  | 2.208256  | 0.220826      | 3        | overhead | #D4834, |

In [67]: draw\_task\_structural\_composition\_donut(fins\_analysis\_df)

```
=====
📊 Task Load Structural Analysis Summary
=====

Total Jobs: 201
Total CPU Time: 204785.00 µs
Number of Algorithm Types: 10

-----
🔗 Task Category Breakdown:
-----

🟦 Critical Path (High Freq + High Load):
  Tasks: n0, n4, n6, n7, n8, n9
  Count: 6
  Total Freq: 60.2%
  Total CPU: 85.2%

🟡 Overhead Source (High Freq + Low Load):
  Tasks: n1, n2, n3, n5
  Count: 4
  Total Freq: 39.8%
  Total CPU: 14.8%

=====
💡 Interpretation:
  • Critical Path: Serial bottleneck per Amdahl's Law
  • Overhead Source: High-frequency small tasks
  • Compute Intensive: Good candidates for parallelization
  • Minor Tasks: Negligible impact on performance
=====
```

Out[67]:

|   | algo | count | P_i       | total_time | C_i_ratio | P_i_times_C_i | sort_idx | category |    |
|---|------|-------|-----------|------------|-----------|---------------|----------|----------|----|
| 0 | n0   | 21    | 10.447761 | 86908      | 42.438655 | 4.433889      | 0        | critical | #2 |
| 4 | n4   | 20    | 9.950249  | 14151      | 6.910174  | 0.687580      | 4        | critical | #2 |
| 6 | n6   | 20    | 9.950249  | 22890      | 11.177576 | 1.112197      | 6        | critical | #2 |
| 7 | n7   | 20    | 9.950249  | 12030      | 5.874454  | 0.584523      | 7        | critical | #2 |
| 8 | n8   | 20    | 9.950249  | 12631      | 6.167932  | 0.613725      | 8        | critical | #2 |
| 9 | n9   | 20    | 9.950249  | 25913      | 12.653759 | 1.259080      | 9        | critical | #2 |
| 1 | n1   | 20    | 9.950249  | 5850       | 2.856655  | 0.284244      | 1        | overhead | #C |
| 2 | n2   | 20    | 9.950249  | 9940       | 4.853871  | 0.482972      | 2        | overhead | #C |
| 3 | n3   | 20    | 9.950249  | 4383       | 2.140293  | 0.212965      | 3        | overhead | #C |
| 5 | n5   | 20    | 9.950249  | 10089      | 4.926630  | 0.490212      | 5        | overhead | #C |

```
In [ ]: import os
import re
import pandas as pd
import numpy as np

def analyze_makespan(root_dir):
    """
    统计每个 CSV 文件的 makespan 信息
    makespan = 当前 wake - 前一次 sleep

    参数:
        root_dir: 数据目录

    返回:
        DataFrame: 每个 seed 一行, 包含 makespan 的统计信息
    """
    pattern = re.compile(
```

```

        r"trace_(?P<kind>\w+)_u(?P<u>\d+)_m(?P<m>\d+)_ms(?P<ms>\d+)_s(?P<seed>\d+)"
    )
    results = []

    if not os.path.exists(root_dir):
        print(f"目录不存在: {root_dir}")
        return pd.DataFrame()

    files = [f for f in os.listdir(root_dir) if f.endswith(".csv")]
    print(f"正在扫描: {root_dir}, 找到 {len(files)} 个 CSV 文件")

    for filename in sorted(files):
        match = pattern.match(filename)
        if match:
            kind = match.group("kind")
            u = int(match.group("u"))
            m = int(match.group("m"))
            ms = int(match.group("ms"))
            seed = int(match.group("seed"))

            path = os.path.join(root_dir, filename)

            # 读取数据
            df = pd.read_csv(path)
            df["t_us"] = pd.to_numeric(df["t_us"], errors='coerce')
            df = df.dropna(subset=["t_us"]).sort_values("t_us")

            # 获取 rollover 事件
            is_rollover = df["tag"].fillna("").str.contains("main::rollover")

            # 获取 wake 和 sleep 事件
            is_wake = df["kind"] == "wake"
            is_sleep = df["kind"] == "sleep"

            # 获取 wake 和 sleep 的时间
            wake_times = df[is_rollover & is_wake]["t_us"].sort_values().values
            sleep_times = df[is_rollover & is_sleep]["t_us"].sort_values().values

            if len(wake_times) == 0 or len(sleep_times) == 0:
                print(f"⚠️ {filename}: wake 或 sleep 事件缺失 (wake={len(wake_times)}, sleep={len(sleep_times)})")
                continue

            # 计算 makespan = 当前 wake - 前一次 sleep
            makespans = []
            for i in range(1, min(len(wake_times), len(sleep_times) + 1)):
                current_wake = wake_times[i]
                prev_sleep = sleep_times[i - 1]
                makespan = current_wake - prev_sleep
                makespans.append(makespan)

            makespans = np.array(makespans)

            results.append({
                "kind": kind,
                "utilization": u,
                "m": m,

```

```

        "ms": ms,
        "seed": seed,
        "hp_count": len(makespans),
        "T_hp_mean": makespans.mean(),
        "T_hp_median": np.median(makespans),
        "T_hp_std": makespans.std(ddof=1),
        "T_hp_var": makespans.var(ddof=1),
        "T_hp_min": makespans.min(),
        "T_hp_max": makespans.max(),
    })

    if not results:
        print("没有提取到任何有效数据")
        return pd.DataFrame()

    return pd.DataFrame(results)

def analyze_rollover(root_dir):
    """
    统计每个 CSV 文件的 rollover 周期信息
    rollover = sleep - wake

    参数:
        root_dir: 数据目录

    返回:
        DataFrame: 每个 seed 一行, 包含 rollover 周期的统计信息
    """
    pattern = re.compile(
        r"trace_(?P<kind>\w+)_u(?P<u>\d+)_m(?P<m>\d+)_ms(?P<ms>\d+)_s(?P<seed>\d+)"
    )
    results = []

    if not os.path.exists(root_dir):
        print(f"目录不存在: {root_dir}")
        return pd.DataFrame()

    files = [f for f in os.listdir(root_dir) if f.endswith(".csv")]
    print(f"正在扫描: {root_dir}, 找到 {len(files)} 个 CSV 文件")

    for filename in sorted(files):
        match = pattern.match(filename)
        if match:
            kind = match.group("kind")
            u = int(match.group("u"))
            m = int(match.group("m"))
            ms = int(match.group("ms"))
            seed = int(match.group("seed"))

            path = os.path.join(root_dir, filename)

            # 读取数据
            df = pd.read_csv(path)
            df["t_us"] = pd.to_numeric(df["t_us"], errors='coerce')
            df = df.dropna(subset=["t_us"]).sort_values("t_us")

```

```

# 获取 rollover 事件
is_rollover = df["tag"].fillna("").str.contains("main::rollover")

# 获取 wake 和 sleep 事件
is_wake = df["kind"] == "wake"
is_sleep = df["kind"] == "sleep"

# 获取 wake 和 sleep 的时间
wake_times = df[is_rollover & is_wake]["t_us"].sort_values().values
sleep_times = df[is_rollover & is_sleep]["t_us"].sort_values().values

if len(wake_times) == 0 or len(sleep_times) == 0:
    print(f" ⚠️ {filename}: wake 或 sleep 事件缺失 (wake={len(wake_times)}, sleep={len(sleep_times)})")
    continue

# 计算 rollover = sleep - wake
min_len = min(len(wake_times), len(sleep_times))
rollover_intervals = []
for i in range(min_len):
    interval = sleep_times[i] - wake_times[i]
    rollover_intervals.append(interval)

intervals = np.array(rollover_intervals)

results.append({
    "kind": kind,
    "utilization": u,
    "m": m,
    "ms": ms,
    "seed": seed,
    "rollover_count": len(intervals),
    "rollover_mean": intervals.mean(),
    "rollover_median": np.median(intervals),
    "rollover_std": intervals.std(ddof=1),
    "rollover_var": intervals.var(ddof=1),
    "rollover_min": intervals.min(),
    "rollover_max": intervals.max(),
    "rollover_q25": np.percentile(intervals, 25),
    "rollover_q75": np.percentile(intervals, 75),
    "rollover_iqr": np.percentile(intervals, 75) - np.percentile(intervals, 25),
})

if not results:
    print("没有提取到任何有效数据")
    return pd.DataFrame()

return pd.DataFrame(results)

# 分析 makespan
df_makespan = analyze_makespan("test")
df_makespan

# 分析 rollover
df_rollover = analyze_rollover("test")
df_rollover

```

```
In [ ]: def calculate_speedup(df_makespan):
        """
        """

        # 排除未能完成调度的数据 (轮次不为 20)
        df_makespan_filtered = df_makespan[df_makespan["hp_count"] == 19].copy()

        if df_makespan_filtered.empty:
            print("⚠ 警告: 没有 hp_count = 20 的数据")
        else:
            print(f"✅ 保留 {len(df_makespan_filtered)} 条 hp_count=20 的数据")

        # 复制数据避免修改原数据
        df = pd.DataFrame()
        df["kind"] = df_makespan_filtered["kind"]
        df["seed"] = df_makespan_filtered["seed"]
        df["u"] = df_makespan_filtered["utilization"]
        df["m"] = df_makespan_filtered["m"]
        df["C"] = df["u"] * df["m"] * 100 * 1000 / 100
        # 假设 C = u% * m * hp(100ms) * 1000, 单位转换为 μs

        # 计算加速比
        df["S"] = (df["C"] / df_makespan_filtered["T_hp_median"])
        df["E"] = (df["S"] / df["m"])

        return df

df_speedup = calculate_speedup(df_makespan)
df_speedup
```

## 1. 交互式加速比曲线 (Science Style)

说明: 绘制  $S(m)$  随  $m$  增长的曲线, 并叠加一条虚线作为 Ideal (Linear) Speedup 参考。

```
In [ ]: # 测试类型颜色映射
        KIND_COLORS = {
            'fork': NATURE_CLASSIC['wakeup_lat'],
            'join': NATURE_CLASSIC['pack'],
            'loop': NATURE_CLASSIC['post'],
            'mixed': NATURE_CLASSIC['sleep_lat'],
            'multihop': NATURE_CLASSIC['Active'],
        }
```